Mobile Demo Script Explanation -- Frontdoor (Exploitation, Challenge 01)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: A hardcoded debug password sits in plain sight in the decompiled bytecode -- call the server directly with it.

Background -- what the terms below mean

What does "Exploitation" mean, as a category? An **Exploitation** challenge gives you a real target Android app with an actual security mistake in it, and asks you to write a second, small app of your own that takes advantage of that mistake to get the flag.

What is a vulnerability? A mistake in how software was built that lets someone do something the developer never intended -- here, get the flag without going through the app the normal, intended way.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**) -- to find the mistake in the first place.

What is a package name? Every Android app has a unique ID string (like org.naija.fdexploit). Android refuses to install two different apps that both use the exact same ID, which occasionally trips up a first submission attempt if the exploit app accidentally reuses the target's own package name.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/01 Frontdoor\Original Target APK.apk
```

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 01 Frontdoor.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

Hardcoded debug backdoor: Flag.getFlag() has a debug branch that substitutes literal credentials (username=testuser, password=passtestuser123) for real input whenever Flag.debug==true. That literal string is a compile-time constant readable straight out of the decompiled bytecode. Our exploit calls the same server endpoint directly with those credentials, bypassing the app's UI entirely. First submission attempt (using the unmodified target APK, package collision) failed with INSTALL_FAILED_ALREADY_EXISTS -- fixed by using a distinct package name (org.naija.fdexploit).

Files to open

1. Open the target APK in jadx-gui:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/01 Frontdoor\Original Target APK.apk

2. MainActivity.java:

C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/01 Frontdoor\Android Studio Project\app\src

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found Flag.getFlag() has a debug branch that substitutes hardcoded credentials (username=testuser, password=passtestuser123) whenever Flag.debug==true -- a literal compile-time constant sitting in the bytecode.

2. I wrote an app that calls the server's own getflag endpoint directly with those credentials, bypassing the app's UI entirely.

3. My first submission attempt used the unmodified target APK and failed with a package-name collision (INSTALL_FAILED_ALREADY_EXISTS); I fixed this by building my exploit under a distinct package name (org.naija.fdexploit).

4. I resubmitted and the platform's analysis log confirmed the flag was returned.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

MOBISEC{noob_hackers_only_check_for_backdoors}

Source: Verbatim from /apk/81741 analysis log

The real, verbatim output this run produced:

```
$ adb shell am start -n org.naija.fdexploit/org.naija.fdexploit.MainActivity
FLAG: MOBISEC{noob_hackers_only_check_for_backdoors}
```